



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



PARALLELISM

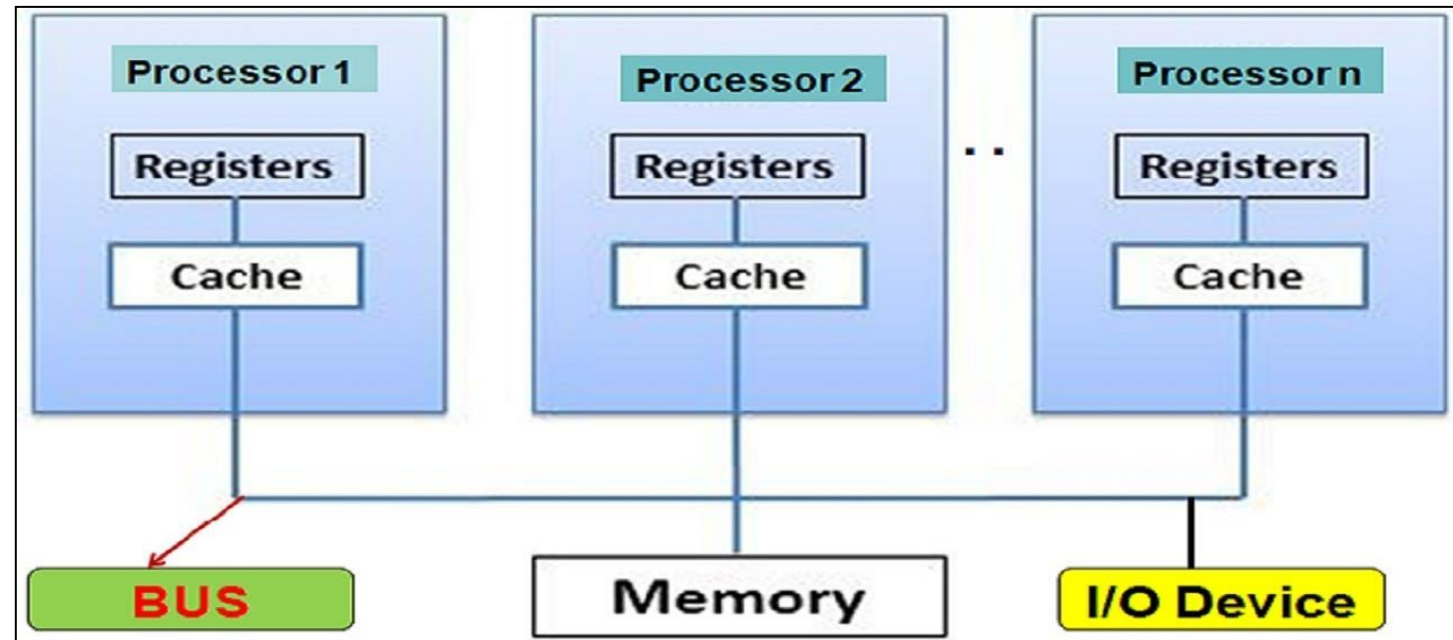
Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Parallelization

- **Parallel** and **distributed systems** have multiple processors that communicate with each other and therefore programming languages used in such systems express concurrency and communication.

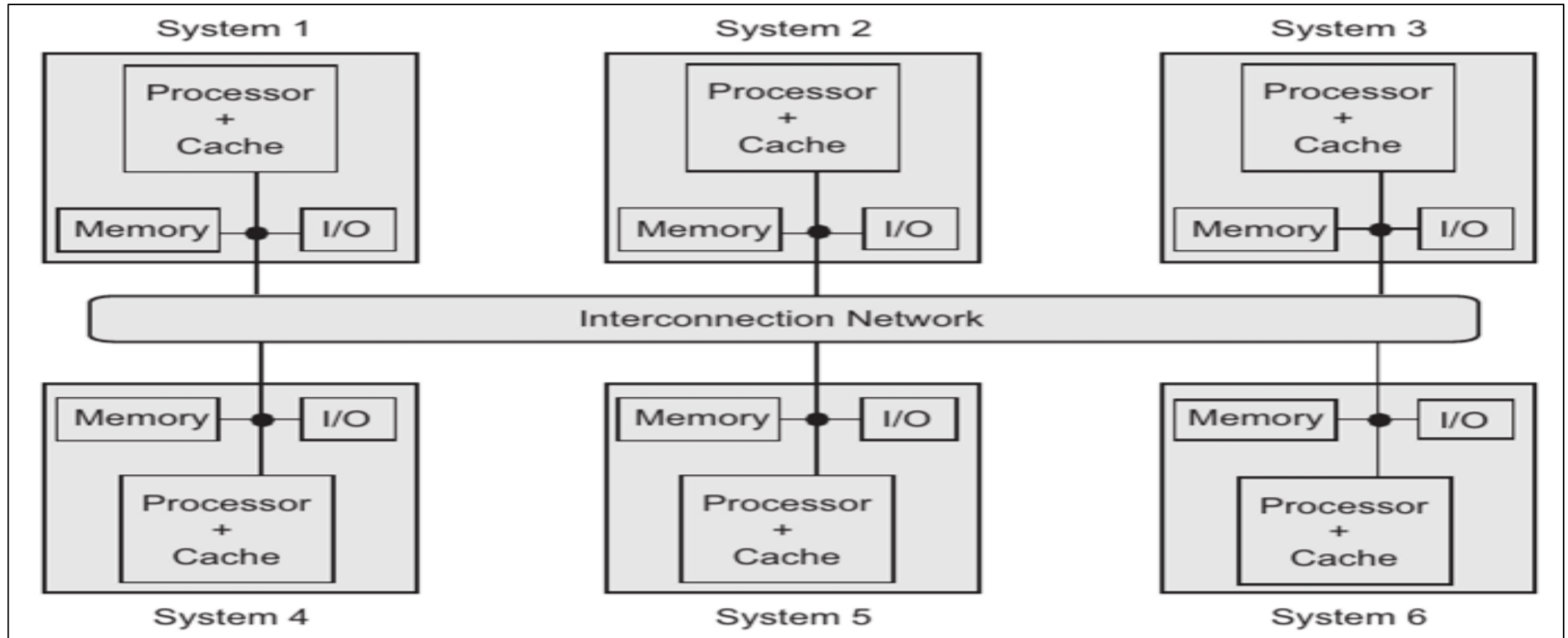
Multiprocessors

- Processes in different processors run in *shared address space*.
- All processors will have access to a single shared memory.
- These processors exchange information by reading and writing variables in the shared memory space.
- To simply construct a multiprocessor we can connect multiple processors onto a **single bus**.
- Modern processors are multiprocessors as they have **multiple cores** which communicate through shared memory/caches.

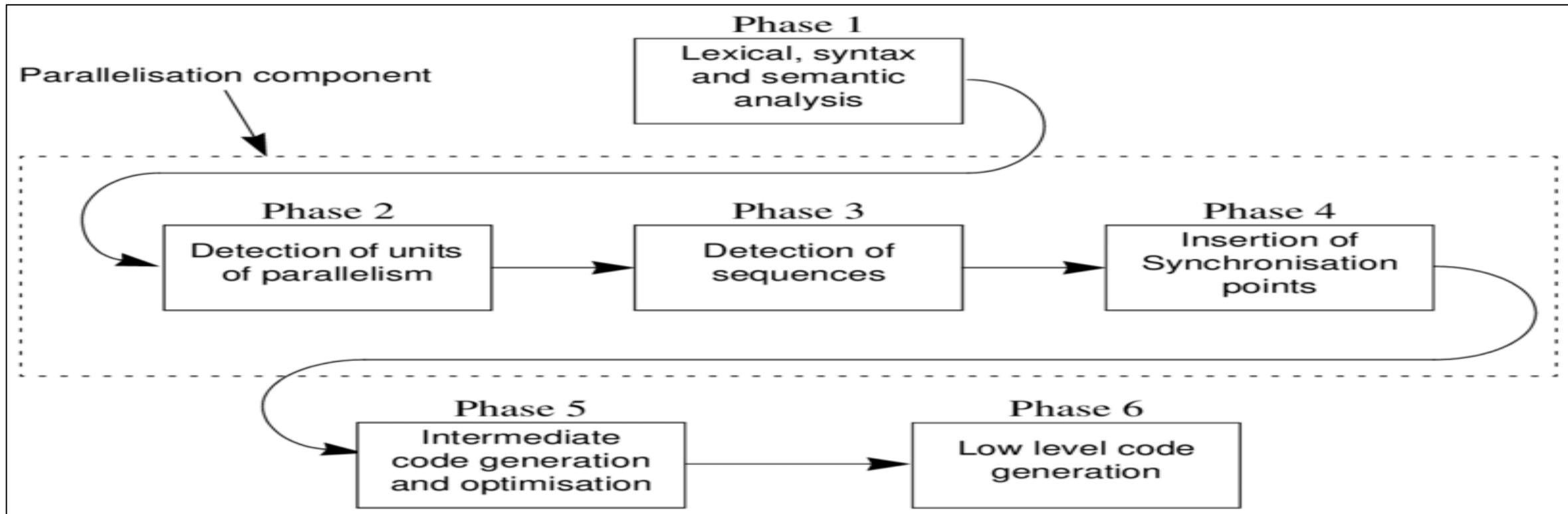


Multicomputer

- These consist of multiple processes which are *connected by a network*.
- Processes in different processors run on *different address space*.
- Processors in a multicomputer communicate by sending messages over the network which can either be custom made for the multicomputer.
- A *cluster* is a multicomputer built of off-the-shelf processors.



- Parallelizing code by hand is very **time consuming**.
- A **parallelizing compiler** will convert your source code into an intermediate representation such as an **Abstract Syntax Tree** (AST) or a **Hierarchical Task Graph** (HTG).
- Then it will perform transformations and optimizations on this intermediate representation in a way that **converts the serial flow of the program into parallel flow**, spreading the work over many processors.



- Then it converts the intermediate representation into parallel code so it can be compiled into a binary.

- The **parallelization task** can be split into three subproblems:
 - *Identifying potential parallelism, which may be done automatically or by a programmer;*
 - *Mapping the parallelism onto the target machine*
 - *Generating and optimizing parallel code.*
- **Parallelizing tools** can be either **semi** or **fully automatic**.
- The parallelization techniques used during the analysis phase in both semi and fully automatic parallelization tools are very similar.
- Semi-automatic tools analyse sequential programs and then provide a partially parallelized solution to the programmer who can then say approve certain parallelized components.
- Fully-automatic tools perform similar parallel analysis but the programmer is not required to provide any input into the final parallelized program.
- Fully automatic tools take the results of the parallelization techniques and create a fully parallelized program from the original sequential version.

Automatic parallelization

- A compiler that automatically converts source code into object code divided into multiple tasks so that more than one process can be executed simultaneously by computer.
- **Automatic parallelization**, also **auto parallelization**, or **autoparallelization** refers to *converting sequential code into multi-threaded and/or vectorized code* in order to use multiple processors simultaneously in a shared-memory multiprocessor (SMP) machine.
- Automatic parallelization *eliminates the need for programmers to manually identify sections of code that are suitable for parallel execution.*
- It also relieves them of the burden of performing data dependency analysis to ensure program correctness and inserting parallel code or directives at relevant locations.
- Parallelism extraction process is normally a *source-to-source transformation*
 - Requires *dependence analysis* to determine the dependence between statements

- **Fully automatic parallelization** of sequential programs is a challenge because it requires *complex program analysis* and the best approach may depend upon parameter values that are not known at compilation time.
- The most important programming control construct for automatic parallelization is the **loop**, since most of the program's execution time happens inside some form of loop.
- There are two main approaches to parallelizing loops:
 - *Pipeline multithreading*
 - *Circular multithreading.*
- For example, consider a loop that applies **100 operations** at each iteration and runs for **1,000 iterations**.
- You can think of this as a 100 column by 1000 row grid with a total of 100,000 operations.
- In circular multithreading, each row is assigned to a different thread.
- In pipeline multithreading, each column is assigned to a different thread

Automatic parallelization **technique**

a. Parse

- This is the first stage where the scanner reads the input source files and identifies all **static** and **external usages**.
- Each line in the file is **separated into tokens** by matching predefined patterns.
- These tokens are saved in a file for later use grammar engine.
- The grammar engine checks patterns of tokens that match predefined rules to identify **variables**, **loops**, and **controls statements**, **functions**, etc. in your code are resumed.

b. Analyze

- Analyzers are used to **identify sections of code** that can be executed concurrently.
- The analyzer uses static data information provided by the scanner parser.
- The analyzer first **finds all completely independent functions** and marks them as separate tasks.
- The analyzer then finds out which tasks have dependencies.

c. Schedule

- The scheduler **lists all tasks** and their **mutual dependencies** in terms of **execution time** and **start time**.
- The scheduler creates an optimal schedule in terms of the number of processors used or the total execution time of the application.

d. Code Generation

- The scheduler generates a list of all tasks along with details on which core the task runs on and how long the task will run.
- The code generator **inserts special constructs** into your code that are read during execution by the scheduler.
- These structures tell the scheduler which core a particular task should run on, along with its start and end times.

Automatic Parallelization tools

YUCCA
Par4All
Cetus
PLUTO

Polaris compiler
AutoPar
iPat/OMP
TRACO
SequenceL

- Consider the following example in which the upper bound of the loop is not known at compile time

```
void loop_test (int u)
{
    for (int i=0; i<u; ++i)
        A[i] = B[i] * C[i];
}
```

- u** could be a small value, the compiler won't automatically parallelize this loop.
- However, you might still want it parallelized because you know that u will always be large

- To enable the auto-parallelization, specify

```
#pragma loop(hint_parallel(n))
```

- n is the number of threads to parallelize across.

```
void loop_test(int u) {
#pragma loop(hint_parallel(8))
    for (int i=0; i<u; ++i)
        A[i] = B[i] * C[i];
}
```

- Compiler will attempt to parallelize the loop across 8 threads.